

CONNECTIONS BETWEEN SCHEDULE-FREE OPTIMIZERS, AdEMAMix, AND ACCELERATED SGD VARIANTS

Anonymous authors
Anonymous submission

ABSTRACT

Recent deep learning optimizers, including Schedule-Free optimizers, AdEMAMix, MARS, and Lion (Chen et al., 2023), were introduced with different motivations, yet they share a consequential structural feature: each changes how the current gradient is separated from, combined with, or weighted against momentum. In parallel, the theory of stochastic acceleration shows that SGD can obtain accelerated behavior in noise-dominated regimes when the momentum coefficient is decoupled from the coefficient assigned to the current gradient. This paper makes that structural correspondence explicit. We derive algebraic rewritings showing that several recent optimizers can be interpreted as accelerated-SGD-style updates, sometimes augmented with preconditioning or weight averaging, and we evaluate the resulting perspective on a 150m-parameter language-modeling task. In the small-batch regime, AdEMAMix, whose update most closely follows the accelerated-SGD template studied here, achieves the strongest performance among the compared optimizers. Guided by the same analysis, we introduce Simplified-AdEMAMix, a one-momentum-buffer variant that matches AdEMAMix in the evaluated small- and large-batch regimes while reducing the state required by the optimizer. The code for Simplified-AdEMAMix will be released upon acceptance.

1 INTRODUCTION

Recent work on deep learning optimization has produced several influential optimizer designs, including Lion (Chen et al., 2023), ScheduleFreeSGD/AdamW (Defazio et al., 2024), and AdEMAMix (Pagliardini et al., 2024). These methods differ in their stated motivations—genetic search, schedule-free training, and multi-timescale exponential averaging—but their update rules share a common structural motif: they modify the relationship between the current stochastic gradient and the momentum term that aggregates past gradients.

A distinct theoretical literature studies acceleration for gradient methods in stochastic, noise-dominated environments. Classical heavy-ball and Nesterov momentum accelerate deterministic gradient descent in important settings, particularly for quadratic objectives, but they do not by themselves accelerate SGD (Jain et al., 2018; Liu & Belkin, 2020). This limitation has led to stochastic momentum schemes that recover acceleration under appropriate assumptions (Jain et al., 2018; Vaswani et al., 2019; Liu & Belkin, 2020; Gupta et al., 2023). Across these schemes, the key algebraic pattern is a decoupling between the coefficient governing momentum persistence and the coefficient multiplying the current gradient in the parameter update.

This paper connects these two developments. We show that Schedule-Free SGD can be rewritten as accelerated SGD followed by weight averaging, and we demonstrate that Lion, Schedule-Free AdamW, MARS, and AdEMAMix admit closely related accelerated-SGD-style interpretations, with additional mechanisms such as preconditioning or output averaging when present in the original algorithms. Prior work has identified relationships between momentum and primal averaging (Defazio, 2021); here we organize several recently proposed optimizers under a single momentum–gradient decoupling view and make explicit which parts of each optimizer are captured by that view.

We examine the empirical implications of this correspondence using a 150m-parameter decoder-only transformer trained on C4. In the small-batch experiment, the model is trained for 15b tokens with a batch size of 32k tokens, emphasizing a high-noise optimization regime. The results are consistent with the algebraic analysis: Schedule-Free AdamW performs similarly to an accelerated-SGD-based AdamW variant (Algorithm 3), accelerated-style variants improve over the AdamW baselines in this setting, and AdEMAMix performs best among the compared methods. Additional large-batch experiments show that some of these advantages do not persist unchanged when the gradient-noise regime changes, highlighting both the usefulness and the limits of the accelerated-SGD interpretation.

The paper makes five contributions:

1. We derive explicit algebraic connections between accelerated SGD and recently proposed optimizers, including Schedule-Free SGD and AdEMAMix.
2. We evaluate these connections on a 150-million-parameter decoder-only transformer by comparing AdamW, Schedule-Free AdamW, AdEMAMix, MARS, and related variants; in the small-batch setting, AdEMAMix performs best among the methods considered.
3. We show that the gains associated with accelerated-style updates are batch-regime dependent in our experiments, diminishing at large batch sizes relative to Adam, and that Adam with momentum scheduling can match AdEMAMix in this regime.
4. We identify a high-batch-size limitation of Schedule-Free AdamW: its coupling of momentum and weight averaging coincides with substantially worse performance than AdamW with cosine decay in the studied setting.
5. We introduce Simplified-AdEMAMix, a one-momentum-buffer modification that matches AdEMAMix across the evaluated small- and large-batch regimes.

2 RELATED WORK

We focus on the literature most directly connected to the paper’s central claim: several modern optimizer designs can be profitably analyzed through the same momentum–gradient decoupling that appears in accelerated SGD.

Jain et al. (2018) introduced an accelerated SGD variant with improved convergence rates for least-squares problems. Kidambi et al. (2018) simplified the corresponding update and showed that standard momentum does not provide acceleration in that setting. Subsequent analyses (Liu & Belkin, 2020; Vaswani et al., 2019; Gupta et al., 2023) extended stochastic acceleration results to convex and strongly convex objectives under different assumptions, thereby establishing a theoretical basis for momentum schemes that differ from the classical heavy-ball form.

Several practical optimizers contain related ingredients. Lucas et al. (2019) proposed an update that forms a weighted sum of multiple momentum terms with distinct coefficients, and Ma & Yarats (2019) developed an optimizer explicitly motivated by the framework of Jain et al. (2018). More recently, Chen et al. (2023) introduced a genetically searched optimizer that weights the gradient and momentum components differently before applying a sign operation, while Pagliardini et al. (2024) proposed an update that blends two momentum scales. These methods motivate the present analysis because apparently heterogeneous optimizer designs can share the same underlying separation between gradient weight and momentum persistence.

3 BACKGROUND

3.1 MOMENTUM

Momentum is the standard mechanism for incorporating past gradient information into the current parameter update. In deterministic optimization, suitable momentum schemes can accelerate gradient descent; in stochastic optimization, however, the relationship between momentum and acceleration is more delicate. For weights w_t and momentum coefficient β , the conventional momentum update is

$$m_t = \beta m_{t-1} + \nabla f(w_t); \quad w_t = w_{t-1} - \eta m_t$$

3.2 WEIGHT AVERAGING

Weight averaging reduces stochastic variability by returning an aggregate of iterates rather than the final iterate alone. Instead of using w_T as the output, the method maintains a weighted average $\bar{w}_T$, with time-dependent averaging weights denoted by γ_t :

$$\bar{w}_T = (1 - \gamma_T)\bar{w}_{T-1} + \gamma_T w_T$$

Throughout the paper, the weight-averaging coefficients take the form

$$\gamma_t \approx 1 - \frac{1}{\delta t}$$

for a constant $0 \leq \delta \leq 1$.

3.3 ACCELERATED SGD

We use the following generalized accelerated SGD template as the common algebraic reference point for the optimizer connections developed below:

$$m_t = \beta_{a,t} m_{t-1} + g_t, \quad w_{t+1} = w_t - \eta_{a,t} m_t - \alpha_{a,t} g_t \quad (1)$$

where $\beta_{a,t}, \alpha_{a,t}, \eta_{a,t}$ are possibly time-dependent scalar coefficients and g_t is the stochastic gradient evaluated at w_t . The subscript ‘a’ denotes quantities associated with this accelerated-SGD parameterization.

Setting $\alpha_{a,t} = 0$ recovers standard SGD with momentum, whereas nonzero $\alpha_{a,t}$ explicitly separates the current-gradient contribution from the momentum coefficient. Many accelerated SGD algorithms in the literature, including those of Jain et al. (2018); Vaswani et al. (2019); Liu & Belkin (2020); Gupta et al. (2023), can be expressed in this template after algebraic reparameterization. Appendix B gives the corresponding reductions.

4 CONNECTIONS BETWEEN EXISTING OPTIMIZERS AND ACCELERATED SGD

This section develops the structural link between accelerated SGD and several recently proposed optimizers. The goal is not to claim that every implementation detail of these methods is identical to the template in Equation (1); rather, the analysis isolates the part of each update that decouples the current-gradient weight from the momentum coefficient. Schedule-Free optimizers, Lion, MARS, and AdEMAMix can all be expressed in this accelerated-SGD-style form, sometimes after accounting for additional preconditioning or output averaging. This view also motivates Simplified-AdEMAMix, which retains one momentum term while matching AdEMAMix in the small- and large-batch experiments reported below.

4.1 SCHEDULE-FREE SGD

Schedule-Free SGD (Defazio et al., 2024) is a constant-learning-rate optimizer designed to remove the need for an explicit learning-rate schedule. Using the notation of Defazio et al. (2024), the update is

$$\begin{aligned} y_t &= (1 - \beta)z_t + \beta x_t \\ z_{t+1} &= z_t - \gamma g(y_t) \\ x_{t+1} &= (1 - c_{t+1})x_t + c_{t+1}z_{t+1} \end{aligned}$$

Here, y_t denotes the model weights at which the gradient is evaluated, while x_t denotes the weights used for evaluation.

We first express the dynamics in terms of y_t and an auxiliary momentum variable m_t , defined as

$$m_{t+1} = \frac{x_t - z_{t+1}}{\gamma}. \quad (2)$$

Expanding this definition gives

$$m_t = \frac{x_t - z_{t+1}}{\gamma} \quad (3)$$

$$= \frac{x_t + \gamma g_t - z_t}{\gamma} \quad (4)$$

$$= \frac{x_t - z_t + \gamma g_t}{\gamma} \quad (5)$$

$$= \frac{(1 - c_t)(x_{t-1} - z_t) + \gamma g_t}{\gamma} \quad (6)$$

$$= (1 - c_t)m_{t-1} + g_t. \quad (7)$$

Thus, m_t obeys the momentum recursion in Equation (1) with $\beta_{a,t} = 1 - c_t$. Substituting this representation into the update for y_t gives

$$y_{t+1} = (1 - \beta)z_{t+1} + \beta x_{t+1} \quad (8)$$

$$= (1 - \beta)(z_t - \gamma g_t) + \beta((1 - c_{t+1})x_t + c_{t+1}z_{t+1}) \quad (9)$$

$$= (1 - \beta)z_t + \beta x_t - (1 - \beta)\gamma g_t + \beta c_{t+1}(z_{t+1} - x_t) \quad (10)$$

$$= y_t - \gamma[\beta c_{t+1}m_t + (1 - \beta)g_t]. \quad (11)$$

Therefore, y_t follows the weight update in Equation (1) with $\eta_{a,t} = \gamma\beta c_{t+1}$, $\alpha_{a,t} = \gamma(1 - \beta)$, and $w_t = y_t$. The training iterate in Schedule-Free SGD is consequently an accelerated-SGD iterate under this reparameterization. Since Schedule-Free SGD evaluates with x_t , the evaluation sequence must also be characterized:

$$\begin{aligned} x_{t+1} &= (1 - c_{t+1})x_t + c_{t+1}z_{t+1} \\ x_{t+1} &= (1 - c_{t+1})x_t + c_2 \left(\frac{y_{t+1} - \beta x_{t+1}}{1 - \beta} \right) \\ x_{t+1}(1 - \beta + c_{t+1}\beta) &= (1 - c_{t+1})(1 - \beta)x_t + c_{t+1}y_{t+1} \\ x_{t+1} &= \frac{(1 - c_{t+1})(1 - \beta)x_t + c_{t+1}y_{t+1}}{(1 - c_{t+1})(1 - \beta) + c_{t+1}}. \end{aligned}$$

This expression shows that x_t is a weighted average of y_t . Recursively expanding the update further shows that, when c_t is constant, x_t is an exponential average of the accelerated iterates. Schedule-Free SGD can therefore be read as accelerated SGD followed by weight averaging, rather than as an unrelated optimization principle.

This decomposition separates two mechanisms that are otherwise intertwined in the original update:

1. The accelerated-SGD component can improve performance relative to standard SGD with momentum in regimes where stochastic acceleration is beneficial.
2. Weight averaging, specifically tailed weight averaging, supplies the schedule-free behavior that permits a constant learning rate; see Section 4.1.3.

The Schedule-Free formulation also provides two implementation advantages for Schedule-Free SGD/Adam:

- It avoids additional memory for an explicit weight-averaged copy of the parameters.
- It removes the explicit weight-averaging coefficient as a separate hyperparameter.

The same coupling becomes restrictive in the large-batch experiments of Section 5.1, where the preferred momentum behavior and the desired averaging window no longer align.

4.1.1 CASE: $\beta = 0.0$

As noted in (Defazio et al., 2024), setting $\beta = 0$ reduces Schedule-Free SGD to standard SGD with weight averaging. Since $c_t = 1/t$, averaging begins at the start of training rather than after a tail period.

4.1.2 CASE: $\beta = 1.0$

As noted in (Defazio et al., 2024), setting $\beta = 1$ reduces Schedule-Free SGD to standard momentum SGD, with momentum coefficient $\beta_{a,t}$ scaling as $1 - 1/t$.

4.1.3 CASE: $\beta = 0.9$

For the default Schedule-Free SGD setting $\beta = 0.9$, the decomposition implies the following effective parameter behavior:

- Since c_t scales as $1/t$, the effective momentum grows as $1 - 1/t$.
- The ratio between the current-gradient weight and the momentum weight is fixed at $(1 - \beta)/(\beta c_{t+1}) \approx 0.11$.
- The resulting weight average covers approximately the most recent 10% of the iterates.

4.2 LION

Lion (Chen et al., 2023) uses the following update:

$$m'_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$\theta_t = \theta_{t-1} - \eta \text{sign}(m'_t)$$

$$m_t = \beta_2 m_{t-1} + (1 - \beta_2) g_t.$$

The first two lines form an accelerated-SGD-style update in which the gradient and momentum contributions are weighted differently before the coordinate-wise sign operation is applied. Lion can therefore be viewed as an accelerated SGD mechanism followed by a sign transformation.

4.3 MARS

The practical version of MARS (Yuan et al., 2024), denoted MARS-Approx, also fits the accelerated SGD framework once its update is rewritten in terms of a shifted momentum variable, with an additional preconditioning step. Ignoring bias correction and clipping, the update is

$$\begin{aligned} c_t &= g_t + \gamma \frac{\beta_1}{1 - \beta_1} [g_t - g_{t-1}] \\ m_t &= \beta_1 m_{t-1} + (1 - \beta_1) c_t \\ v_t &= \beta_2 v_{t-1} + (1 - \beta_2) c_t^2 \\ x_{t+1} &= x_t - \eta \frac{m_t}{\sqrt{v_t} + \epsilon} \end{aligned}$$

where m_t and v_t are the first- and second-order moment estimates and x_t denotes the model parameters. Introducing $\hat{m}_t = m_t - \gamma g_t$ gives

$$\begin{aligned} c_t &= g_t + \gamma \frac{\beta_1}{1 - \beta_1} [g_t - g_{t-1}] \\ \hat{m}_t &= \beta_1 \hat{m}_{t-1} + (1 - \beta_1)(1 - \gamma) g_t \\ v_t &= \beta_2 v_{t-1} + (1 - \beta_2) c_t^2 \\ x_{t+1} &= x_t - \eta \frac{\hat{m}_t + \gamma g_t}{\sqrt{v_t} + \epsilon} \end{aligned}$$

In this form, the momentum recursion follows the accelerated SGD template, while the denominator retains the MARS-specific preconditioning rule rather than the AdamW preconditioner. Section 5 evaluates this optimizer alongside the other accelerated-style variants.

4.4 ADEMAMIX

AdEMAMix (Pagliardini et al., 2024) is structurally close to accelerated-SGD-based AdamW. The main distinction is representational: accelerated SGD combines the current gradient with a single momentum term, whereas AdEMAMix maintains two momentum terms with different coefficients and combines them linearly in the update. Algorithm 1 states the algorithm.

For analysis, we consider the AdEMAMix variant with $\beta_1 = 0$. Pagliardini et al. (2024) reports that this simplified version is nearly equivalent to the full optimizer for small batch sizes, and our experiments in Section 5 confirm the same pattern. With $\beta_1 = 0$, AdEMAMix aligns with the accelerated SGD form in Equation (1). Moreover, the prescribed schedules for β_3 , the momentum coefficient, and α , which controls the relative current-gradient weight, closely match theoretical schedules for accelerated SGD (Gupta et al., 2023).

In smooth convex optimization, achieving acceleration in stochastic settings requires a momentum scheme of the form:

$$\beta_{a,t} = 1 - \frac{k}{t}$$

for some constant $k > 0$, as established by Gupta et al. (2023). AdEMAMix approximates this schedule by increasing β_3 over training.

The comparison is slightly obscured by the normalization convention for the momentum buffer. In accelerated SGD, momentum is maintained as

$$m_t = \beta_{a,t} m_{t-1} + g_t$$

whereas Algorithm 1 uses

$$m_2^{(t)} \leftarrow \beta_3^{(t)} m_2^{(t-1)} + (1 - \beta_3^{(t)}) g^{(t)}.$$

When $\beta_{a,t}$ scales as $1 - 1/t$, the accumulated contribution of past gradients in the accelerated SGD buffer grows proportionally to t . AdEMAMix compensates for the EMA normalization through α , which also scales proportionally to t . This close structural match explains why AdEMAMix is the strongest method in the small-batch comparison in Figure 1.

At large batch sizes, however, AdEMAMix with $\beta_1 = 0.0$ exhibits the performance degradation reported by Pagliardini et al. (2024). The theory of Gupta et al. (2023) indicates that the weight assigned to the current gradient should decrease as batch size grows. AdEMAMix instead fixes the current-gradient weight at 1, which likely accounts for the degradation observed in the large-batch regime.

The next section uses this diagnosis to define a simplified AdEMAMix variant that includes an explicit current-gradient weight, removes the need to schedule α , and maintains only one momentum term. The experiments show that this simplification preserves AdEMAMix-level performance in both evaluated batch-size regimes.

4.5 SIMPLIFIED ADEMAMIX

Building on the insights discussed above, we propose a simplified optimizer that eliminates the need for maintaining two separate momentum terms and removes the requirement for scheduling α . The optimizer is formally presented in Algorithm 2, where we employ theory-style momentum (instead of the exponential moving average (EMA) style). In the final update, we assign a fixed weight α to the gradient. We note that setting $\alpha = 0$ recovers the standard Adam optimizer (subject to appropriate transformations of η and β_1). In Section 5, we demonstrate that this simplified variant matches the performance of AdEMAMix across both small and large batch sizes.

Algorithm 1 Single step of AdEMAMix optimizer.

- 1: **Input:** Data distribution $\mathcal{D}$. Initial model parameters $\theta^{(0)}$. Number of iterations T . Learning rate η . ϵ a small constant. AdamW parameters: β_1, β_2 . AdEMAMix parameters β_3, α . Warmup parameter T_{α, β_3} , note that we usually set it to T . β_{start} is usually set to β_1 .
 - 2: Optional: use schedulers $\eta^{(t)}, \beta_3^{(t)} \leftarrow f_{\beta_3}(t, \beta_3, \beta_{\text{start}}, T_{\alpha, \beta_3})$ and $\alpha^{(t)} \leftarrow f_{\alpha}(t, \alpha, T_{\alpha, \beta_3})$
 - 3: Sample batch: $x \sim \mathcal{D}$
 - 4: Compute gradient: $g^{(t)} \leftarrow \nabla_{\theta} \mathcal{L}_{\theta^{(t-1)}}(x)$
 - 5: Update the fast EMA m_1 : $m_1^{(t)} \leftarrow \beta_1 m_1^{(t-1)} + (1 - \beta_1) g^{(t)}$
 - 6: Update the slow EMA m_2 : $m_2^{(t)} \leftarrow \beta_3^{(t)} m_2^{(t-1)} + (1 - \beta_3^{(t)}) g^{(t)}$
 - 7: Update the second moment estimate: $\nu^{(t)} \leftarrow \beta_2 \nu^{(t-1)} + (1 - \beta_2) (g^{(t)})^2$
 - 8: Update parameters: $\theta^{(t)} \leftarrow \theta^{(t-1)} - \eta^{(t)} \left(\frac{\hat{m}_1^{(t)} + \alpha^{(t)} m_2^{(t)}}{\sqrt{\hat{\nu}^{(t)} + \epsilon}} \right)$
-

Algorithm 2 Single step of Simplified AdEMAMix optimizer.

- 1: **Input:** Data distribution $\mathcal{D}$. Initial model parameters $\theta^{(0)}$. Number of iterations T . Learning rate η . ϵ a small constant. AdamW parameters: β_1, β_2 . AdEMAMix parameters $\alpha, \beta_{\text{start}}$. Warmup parameter T_{β_1} , note that we usually set it to T .
 - 2: Optional: use schedulers $\eta^{(t)}, \beta_1^{(t)} \leftarrow f_{\beta_1}(t, \beta_1, \beta_{\text{start}}, T_{\beta_1})$
 - 3: Sample batch: $x \sim \mathcal{D}$
 - 4: Compute gradient: $g^{(t)} \leftarrow \nabla_{\theta} \mathcal{L}_{\theta^{(t-1)}}(x)$
 - 5: Update the EMA m_1 : $m_1^{(t)} \leftarrow \beta_1 m_1^{(t-1)} + g^{(t)}$
 - 6: Update the second moment estimate: $\nu^{(t)} \leftarrow \beta_2 \nu^{(t-1)} + (1 - \beta_2) (g^{(t)})^2$
 - 7: Update parameters: $\theta^{(t)} \leftarrow \theta^{(t-1)} - \eta^{(t)} \left(\frac{m_1^{(t)} + \alpha g^{(t)}}{\sqrt{\hat{\nu}^{(t)} + \epsilon}} \right)$
-

Algorithm 3 Single step of accelerated SGD based Adam with weight averaging. For simplicity we ignore the initialization, other boundary effects such as bias correction, and weight decay. Hyperparameters: Learning rate η , betas = $(\beta_1, \beta_2, \beta_3)$, weight averaging coefficient δ , and epsilon ϵ .

-
- 1: Sample batch B_t .
 - 2: $g \leftarrow -\nabla_w \phi_{B_t}(w_t)$
 - 3: $v \leftarrow \beta_2 v + (1 - \beta_2)(g \odot g)$
 - 4: $N \leftarrow \frac{\beta_3 m + (1 - \beta_3)g}{\sqrt{\hat{v}} + \epsilon}$
 - 5: $w \leftarrow w - \eta N$
 - 6: $m \leftarrow \beta_1 m + (1 - \beta_1)g$
 - 7: $c = \max(1 - 1/t, 1 - 1/(\delta t))$
 - 8: $w_{\text{avg}} \leftarrow c w_{\text{avg}} + (1 - c)w$
-

5 EXPERIMENTS

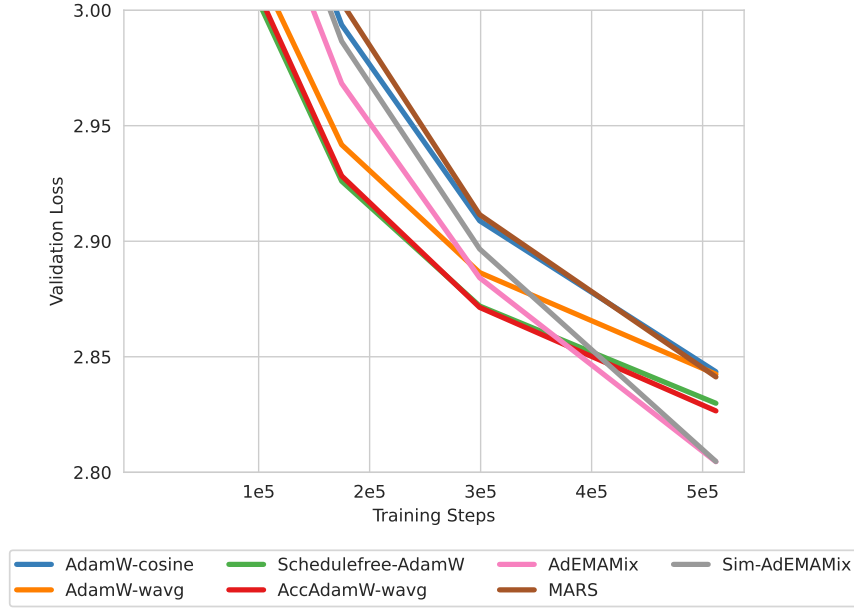


Figure 1: Comparison of the best runs of various optimizers as stated in Section 5 for language modeling task on a decoder-only 150m transformer model. We find that AdEMAMix and simplified-AdEMAMix perform the best, owing to their precise similarity to accelerated SGD variants.

In this section, we present experiments conducted on a 150-million-parameter decoder-only transformer model for a language modeling task using the C4 dataset. The model is trained with a sequence length of 1024 and a batch size of 32, over 15 billion tokens ($\approx 5 \times$ Chinchilla), ensuring that the training operates in a noise-dominated regime.

We compare the following optimization algorithms:

1. Standard AdamW with cosine decay
2. Standard AdamW with weight averaging
3. Schedule-Free AdamW
4. Accelerated AdamW with weight averaging (Algorithm 3)
5. MARS
6. AdEMAMix

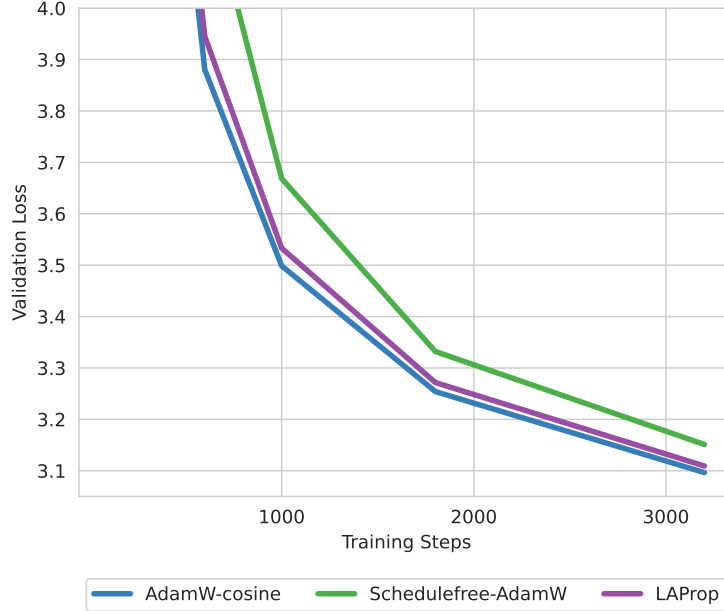


Figure 2: Comparison of the best runs of AdamW with cosine decay, schedule free AdamW and LARProp at higher batch size. Experimental details can be found in Section 5.1

7. Simplified-AdEMAMix

Details of hyperparameter sweeps for these algorithms are provided in Appendix A.

As illustrated in Figure 1, Schedule-Free AdamW and Accelerated AdamW with tailed weight averaging perform comparably, supporting our theoretical claims. Furthermore, both outperform AdamW with cosine decay and AdamW with tailed weight averaging. Moreover, AdEMAMix and Simplified-AdEMAMix outperform all methods, which we hypothesize is due to their alignment with accelerated SGD variants.

5.1 LARGE BATCH SIZE EXPERIMENTS

While the previous experiments focused on the small batch size regime (i.e., training with noisy gradients), we now conduct experiments in the large batch size regime to assess whether these algorithms generalize effectively. In this setup, we train models with a batch size of 1 million tokens over 3 billion tokens ($\approx$ Chinchilla scale).

Schedule-Free AdamW: As shown in Figure 2, Schedule-Free AdamW performs significantly worse compared to AdamW. We attribute this performance gap to the coupling between weight averaging and momentum coefficients. At higher batch sizes, the optimal momentum value is significantly lower than $1 - 1/t$. Although one could use a scaling factor $\approx 1 - r/t$ for some $r \geq 1$, a higher r reduces the effective weight averaging window.

Another key distinction between AdamW and Schedule-Free AdamW is the order in which momentum and preconditioning are applied. AdamW applies momentum before preconditioning, whereas Schedule-Free AdamW applies preconditioning before momentum, making it algorithmically similar to LARProp (Ziyin et al., 2021). However, as shown in Figure 2, the performance of AdamW is comparable to that of LARProp, suggesting that this difference is not the primary cause of the performance gap.

AdEMAMix: For large batch sizes, as previously observed in Pagliardini et al. (2024), Figure 3 shows that setting $\beta_1 = 0.0$ in AdEMAMix results in a significant performance drop compared to using two separate momentum terms. This degradation occurs because AdEMAMix assigns a fixed weight of 1 to the current gradient, whereas theoretical

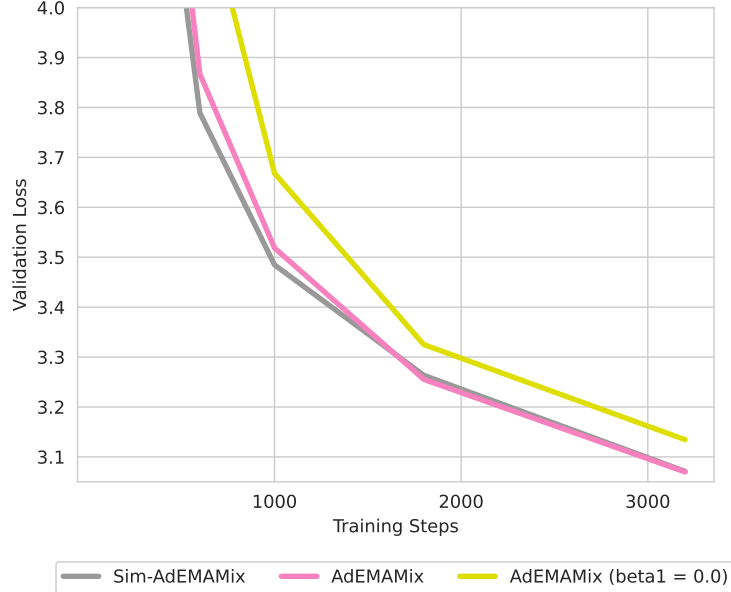


Figure 3: Comparison of the best runs of AdEMAMix (with and without $\beta_1 = 0.0$) and our variant of simplified AdEMAMix for higher batch size experiments. Experimental details can be found in Section 5.1

accelerated SGD variants (Gupta et al., 2023) require a diminishing weight on the current gradient as batch size increases.

Additionally, as depicted in Figure 3, our proposed variant, Simplified-AdEMAMix, achieves performance equivalent to AdEMAMix while eliminating the need for two separate momentum terms. Notably, we achieve this performance at $\alpha = 0.0$, meaning Simplified-AdEMAMix reduces to standard Adam with momentum scheduling.

6 CONCLUSION

In this work, we establish explicit connections between accelerated SGD variants and several recently proposed optimizers, including Schedule-Free optimizers, AdEMAMix, MARS, and Lion. We also present empirical evidence demonstrating that AdEMAMix, which aligns most closely with theoretical accelerated SGD variants, achieves superior performance in small batch size training.

Building on this connection, we introduce Simplified-AdEMAMix, which removes the need for maintaining two separate momentum buffers. We empirically show that Simplified-AdEMAMix matches the performance of AdEMAMix across both small and large batch sizes while eliminating the additional memory overhead associated with AdEMAMix.

REFERENCES

- Chen, X., Liang, C., Huang, D., Real, E., Wang, K., Pham, H., Dong, X., Luong, T., Hsieh, C., Lu, Y., and Le, Q. V. Symbolic discovery of optimization algorithms. In Oh, A., Naumann, T., Globerson, A., Saenko, K., Hardt, M., and Levine, S. (eds.), *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*, 2023. URL http://papers.nips.cc/paper_files/paper/2023/hash/9a39b4925e35cf447ccba8757137d84f-Abstract-Conference.html.
- Defazio, A. Momentum via primal averaging: Theoretical insights and learning rate schedules for non-convex optimization, 2021. URL <https://arxiv.org/abs/2010.00406>.

- Defazio, A., Yang, X., Mehta, H., Mishchenko, K., Khaled, A., and Cutkosky, A. The road less scheduled. *CoRR*, abs/2405.15682, 2024. doi: 10.48550/ARXIV.2405.15682. URL <https://doi.org/10.48550/arXiv.2405.15682>.
- Gupta, K., Siegel, J., and Wojtowysch, S. Achieving acceleration despite very noisy gradients, 2023. URL <https://arxiv.org/abs/2302.05515>.
- Jain, P., Kakade, S. M., Kidambi, R., Netrapalli, P., and Sidford, A. Accelerating stochastic gradient descent for least squares regression. In Bubeck, S., Perchet, V., and Rigollet, P. (eds.), *Proceedings of the 31st Conference On Learning Theory*, volume 75 of *Proceedings of Machine Learning Research*, pp. 545–604. PMLR, 06–09 Jul 2018. URL <https://proceedings.mlr.press/v75/jain18a.html>.
- Kidambi, R., Netrapalli, P., Jain, P., and Kakade, S. M. On the insufficiency of existing momentum schemes for stochastic optimization. In *International Conference on Learning Representations*, 2018. URL <https://openreview.net/forum?id=rJTutzbA->.
- Liu, C. and Belkin, M. Accelerating sgd with momentum for over-parameterized learning. In *International Conference on Learning Representations*, 2020. URL <https://openreview.net/forum?id=rlgixp4FPH>.
- Lucas, J., Sun, S., Zemel, R., and Grosse, R. Aggregated momentum: Stability through passive damping. In *International Conference on Learning Representations*, 2019. URL <https://openreview.net/forum?id=Syxt5oC5YQ>.
- Ma, J. and Yarats, D. Quasi-hyperbolic momentum and adam for deep learning. In *International Conference on Learning Representations*, 2019. URL <https://openreview.net/forum?id=SlfUpoR5FQ>.
- Pagliardini, M., Ablin, P., and Grangier, D. The ademamix optimizer: Better, faster, older. 2024. URL <https://arxiv.org/abs/2409.03137>.
- Porian, T., Wortsman, M., Jitsev, J., Schmidt, L., and Carmon, Y. Resolving discrepancies in compute-optimal scaling of language models. *CoRR*, abs/2406.19146, 2024. doi: 10.48550/ARXIV.2406.19146. URL <https://doi.org/10.48550/arXiv.2406.19146>.
- Vaswani, S., Bach, F., and Schmidt, M. Fast and faster convergence of sgd for over-parameterized models and an accelerated perceptron. In Chaudhuri, K. and Sugiyama, M. (eds.), *Proceedings of the Twenty-Second International Conference on Artificial Intelligence and Statistics*, volume 89 of *Proceedings of Machine Learning Research*, pp. 1195–1204. PMLR, 16–18 Apr 2019. URL <https://proceedings.mlr.press/v89/vaswani19a.html>.
- Yuan, H., Liu, Y., Wu, S., Zhou, X., and Gu, Q. Mars: Unleashing the power of variance reduction for training large models, 2024. URL <https://arxiv.org/abs/2411.10438>.
- Ziyin, L., Wang, Z. T., and Ueda, M. Laprop: Separating momentum and adaptivity in adam, 2021. URL <https://arxiv.org/abs/2002.04839>.

A HYPERPARAMETERS

Below are the hyperparameters for the small batch experiments.

1. AdamW with cosine decay - 51.2k warmup - learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, β_1 in $[0.9, 0.95]$, β_2 in $[0.99, 0.999, 0.99968, 0.9999]$. The optimal values of β_1 and β_2 were .9 and .999 respectively matching the default values. We note that for larger batch sizes it is common to use $\beta_2 = .95$, the benefit of higher β_2 at smaller batch sizes has also been observed by Porian et al. (2024).
2. AdamW with cosine decay - 10k warmup - learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, $\beta_1 = 0.9$, $\beta_2 = 0.999$ i.e. we fix β_1, β_2 to be the optimal values from the previous sweep. This performed worse than warmup of 51.2k steps.
3. AdamW constant fraction weight averaging: - learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, $\beta_1 = 0.9$, β_2 in $[0.99, 0.997, 0.999, 0.9997]$, δ in $[0.05, 0.1, 0.2]$.
4. AdamW with cosine decay and weight averaging - learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, δ in $[0.025, 0.05, 0.1]$.
5. Accelerated SGD based AdamW with cosine decay - learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, β_1 in $[0.999, 0.99968, 0.9999]$, β_2 in $[0.99, 0.9968, 0.999]$, $\beta_3 = 0.9$
6. Accelerated SGD based AdamW with constant learning rate and weight averaging - learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, β_1 in $[0.99684, 0.999]$, β_2 in $[0.999]$, $\beta_3 = 0.9$, δ in $[0.05, 0.1]$
7. Accelerated SGD based AdamW with cosine decay and weight average - learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, β_1 in $[0.99684, 0.999]$, $\beta_2 = 0.999$, δ in $[0.05, 0.1]$, $\beta_3 = 0.9$
8. Schedulefree AdamW with constant learning rate - learning rate in $[3.16e-4, 1e-3, 3.16e-3, 1e-2]$, β_1 in $[0.8, 0.9, 0.95]$, $\beta_2 = 0.999$
9. Schedulefree AdamW with cosine decay - $[3.16e-4, 1e-3, 3.16e-3, 1e-2]$, β_1 in $[0.8, 0.9, 0.95]$, $\beta_2 = 0.999$
10. MARS - $[3.16e-4, 1e-3, 3.16e-3, 1e-2]$, β_1 in $[0.9, 0.95, 0.99]$, β_2 in $[0.99, 0.999]$, γ in $[0.0, 0.01, 0.02, 0.03, 0.04, 0.05]$, precondition 1d was set to True.
11. AdEMAMix - $[3.16e-4, 1e-3, 3.16e-3]$, β_1 in $[0.0, 0.9]$, $\beta_2 = 0.999$, β_3 in $[0.99, 0.999, 0.9999]$, α in $[2, 4, 8, 16]$.
12. Sim-AdEMAMix - $[1e-6, 3.16e-6, 1e-5, 3.16e-5]$, β_1 in $[0.99, 0.999, 0.9999]$, $\beta_2 = 0.999$, α in $[10, 20, 50, 100]$

The hyperparameter sweeps for the large batch experiments are provided below:

1. Schedule-Free AdamW: $[1e-3, 3.16e-3, 1e-2]$, β in $[0.8, 0.9, 0.95]$, β_2 in $[0.9, 0.95]$, r in $[0.0, 5.0, 9.0, 50.0]$
2. AdamW: $[1e-3, 3.16e-3, 1e-2]$, β_1 in $[0.9, 0.95]$, β_2 in $[0.9, 0.95]$
3. LAMProp: $[1e-3, 3.16e-3, 1e-2]$, β_1 in $[0.9, 0.95]$, β_2 in $[0.9, 0.95]$
4. AdEMAMix: $[1e-3, 3.16e-3, 1e-2]$, β_1 in $[0.0, 0.9]$, $\beta_2 = 0.95$, β_3 in $[0.9, 0.95, 0.99]$, α in $[2, 4, 8, 16]$
5. Sim-AdEMAMix: $[1e-4, 3.16e-4, 1e-3]$, β_1 in $[0.9, 0.95, 0.99]$, $\beta_2 = 0.95$, α in $[0.0, 0.5, 1.0]$

B EQUIVALENCE OF PREVIOUS ACCELERATION METHODS

The general accelerated SGD form is provided in Equation (1). In this section, we will show that all the methods in the works Jain et al. (2018); Vaswani et al. (2019); Liu & Belkin (2020); Gupta et al. (2023) fall within this form.

B.1 AGNES

The update for Gupta et al. (2023) is given below:

$$x'_n = x_n + \alpha v_n \quad x_{n+1} = x'_n - \eta g'_n \quad v_{n+1} = \rho_n(v_n - g'_n)$$

where g'_n is stochastic gradient evaluated on x'_n and the final function is evaluated on x_n . The above equations can be rewritten as

$$x'_{n+1} = x'_n - \eta g'_n + \alpha v_{n+1} \quad - \frac{v_{n+1}}{\rho_n} = \rho_{n-1} \left(- \frac{v_n}{\rho_{n-1}} \right) + g'_n$$

Thus x'_{n+1} follows update equation of the form of Equation (1).

B.2 ASGD

The update for Jain et al. (2018) is given by:

$$y_{j-1} = \alpha x_{j-1} + (1 - \alpha)v_{j-1} \quad x_j = y_{j-1} - \delta g_{j-1} \quad z_{j-1} = \beta y_{j-1} + (1 - \beta)v_{j-1} \quad v_j = z_{j-1} - \gamma g_{j-1}$$

where g_{j-1} represents the stochastic gradient evaluated on y_{j-1} and the function is evaluated on the tail averaged x .

The update equations above can be rewritten as:

$$y_j = y_{j-1} - \alpha \delta g_{j-1} - (1 - \alpha)[y_{j-1} - v_j] \quad \frac{y_{j-1} - v_j}{\gamma - (1 - \beta)\alpha\delta} = (1 - \beta)\alpha \frac{y_{j-2} - v_{j-1}}{\gamma - (1 - \beta)\alpha\delta} + g_{j-1}$$

The update equations above follow the form of Equation (1).

B.3 MASS

The update for Liu & Belkin (2020) is given by:

$$w_{t+1} = u_t - \eta_1 g_t \quad u_{t+1} = (1 + \gamma)w_{t+1} - \gamma w_t + \eta_2 g_t$$

where g_t is the stochastic gradient evaluated on u_t and the function is evaluated on w_t . These equations can be rewritten as:

$$u_{t+1} = u_t - \gamma(w_t - u_t) - [\eta_1(1 + \gamma) - \eta_2]g_t \quad \frac{w_t - u_t}{\eta_1\gamma - \eta_2} = \gamma \frac{w_{t-1} - u_{t-1}}{\eta_1\gamma - \eta_2} + g_t$$

The update equations above follow the form of Equation (1).

B.4 SGD WITH NESTEROV ACCELERATION

The update for Vaswani et al. (2019) is given by:

$$w_{k+1} = \zeta_k - \eta g_k \quad \zeta_k = \alpha_k v_k + (1 - \alpha_k)w_k \quad v_{k+1} = \beta_k v_k + (1 - \beta_k)\zeta_k - \gamma_k \eta g_k$$

These equations can be rewritten as:

$$\zeta_{k+1} = \zeta_k - \eta g_k + \alpha_{k+1}[v_{k+1} - w_{k+1}]; \quad v_{k+1} - w_{k+1} = \beta_k(1 - \alpha_k)[v_k - w_k] - \eta(\gamma_k - 1)g_k$$